SKILLARENA AI

Full Project Plan & Technical Blueprint

AI Interview Preparation • Adaptive Learning • Gamification

React + Vite | Python FastAPI | Pandas | NumPy | Scikit-learn | MongoDB | Redis

Complete consolidated blueprint covering product context, modules, HLD/LLD, database, APIs, UI/UX, ML, gamification, caching, security, deployment, roadmap, and evaluation.

Contents

1. 1. Project Overview
2. 2. Problem Statement and Opportunity
3. 3. Technology Stack
4. 4. Functional Modules
5. 5. Answer Evaluation and Rating
6. 6. Machine Learning and Adaptive Analytics
7. 7. Gamification
8. 8. Redis Caching and Optimization
9. 9. Anonymous Access, Persistence, and Privacy
10. 10. High-Level Design (HLD)
11. 11. Low-Level Design (LLD)
12. 12. MongoDB Database Schema
13. 13. API Design (REST / Postman Friendly)
14. 14. Frontend UI/UX Plan
15. 15. Exact Directory Structure
16. 16. AI Prompt Blueprint
17. 17. Security and Performance
18. 18. Deployment Architecture
19. 19. Eight-Week Implementation Roadmap
20. 20. Success Metrics and Test Plan
21. 21. MVP Scope and Priorities
22. 22. Product Journey and Final Vision

1. Project Overview

SkillArena AI is an AI-powered interview preparation and adaptive learning platform. A user supplies a topic, notes, or learning material. The system analyzes the content, generates grounded questions, evaluates submitted answers, explains mistakes, gives a score and rating, recommends improvements, and adapts future practice based on performance.

The platform is designed for students, freshers, interview candidates, and self-learners. No mandatory login or registration is required. Its central value is to identify knowledge gaps and turn them into a personalized practice path.

- Generate MCQ, descriptive, technical, scenario-based, and HR questions.
- Assess answers for correctness, concept coverage, explanation quality, and application.
- Provide corrected answers, missing concepts, actionable suggestions, scores, and ratings.
- Track performance, topic mastery, accuracy, response time, and repeated mistakes.
- Use XP, levels, badges, streaks, daily quests, combos, skill trees, and optional leaderboards.
- Use MongoDB for persistent records and Redis for caching, rate limits, locks, and leaderboard data.

Item	Description
Product name	SkillArena AI
Product type	AI interview preparation and adaptive learning platform
Target users	Students, freshers, interview candidates, self-learners
Access	Anonymous-first; no required account
Core differentiator	Content-grounded answer evaluation with adaptive practice and gamification

2. Problem Statement and Opportunity

Learners often face practical skill gaps, career confusion, limited mentorship, low interview confidence, curriculum-to-job mismatch, and little actionable feedback. Job seekers may struggle with experience requirements, resume screening, competition, and tracking their preparation.

Companies may face high application volumes, costly screening, difficulty validating skills, slow hiring, fresher training costs, and onboarding challenges. SkillArena AI's initial scope is learning and preparation—not a full recruitment marketplace.

- Student challenges: weak practical experience, unclear career direction, limited mentoring, and expensive resources.
- Job seeker challenges: interview anxiety, skill mismatch, no useful rejection feedback, and limited entry-level opportunities.
- Employer challenges: time-consuming screening, skill verification, hiring costs, and training needs.
- Opportunity: provide structured practice and measurable evidence of topic-level learning.

3. Technology Stack

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	Responsive web application
UI styling	Tailwind CSS	Reusable design system
Animation	Motion / Framer Motion	Feedback and reward effects
Client state	Zustand	Session and UI state
API state	TanStack Query	Fetching, retries, request caching
Charts	Recharts	Analytics visualization
Browser persistence	IndexedDB; localStorage for identifier	Anonymous progress
Backend	Python + FastAPI	REST API and business logic
Validation	Pydantic	Typed validation and structured output
Data analysis	Pandas + NumPy	Statistics and feature engineering
ML	Scikit-learn; Bayesian Knowledge Tracing	Mastery and adaptive prediction
Database	MongoDB	Durable content and attempt records
Cache	Redis	Cache, rate limit, locks, leaderboard
AI provider	Backend provider adapter/SDK	Question generation and evaluation
Deployment	Docker + CDN/cloud hosting	Repeatable deployment
Monitoring	Sentry + structured logs	Error tracking and observability

Pandas and NumPy are analysis libraries, not AI models. Begin with rules and mastery tracking. Train ML models only after collecting sufficient reliable, consented, appropriately anonymized data.

4. Functional Modules

Module	Responsibilities
Content Input & Analyzer	Accept topic/text/PDF/DOCX; extract and clean text; chunk material; identify concepts and subtopics; content hashing.
Question Generator	Generate grounded questions by type and difficulty; produce answer keys and rubrics; detect duplicates.
Answer Evaluation Agent	Evaluate semantic correctness and concept coverage; identify errors and gaps; return corrected answer and suggestions.
Performance Analytics	Compute accuracy, score trends, response time, strong/weak topics, and recurring mistakes.
Gamification Engine	Manage XP, levels, badges, quests, streaks, combo rewards, personal bests, and leaderboard points.
Adaptive Learning Engine	Choose next topic, question, difficulty, and revision interval based on learner performance.
Anonymous Session & Storage	Create anonymous browser ID, maintain practice sessions, save locally, and optionally sync anonymous records.

Recommended architecture is a modular monolith for the MVP. This keeps implementation, testing, and deployment manageable; modules can be split into services only when justified by scale.

5. Answer Evaluation and Rating

Evaluation should use the question, source content, reference answer, and rubric. Assess meaning rather than exact wording. If the source is ambiguous or insufficient, the evaluator must disclose uncertainty rather than invent facts or confidently mark an answer wrong.

Descriptive criterion	Weight
Conceptual correctness	40%
Key concept coverage	25%
Explanation quality	15%
Practical application	10%
Clarity and structure	10%
Total	100%

Score	Rating	Interpretation
90–100	Excellent	Accurate, complete, well explained
75–89	Good	Mostly correct; minor gaps
50–74	Needs Improvement	Partially correct; concepts missing
0–49	Weak	Incorrect or substantially incomplete

Coding rubric: output correctness 40%, algorithmic logic 25%, edge cases 15%, time/space complexity 10%, and code quality 10%. Prefer deterministic code execution and tests over LLM-only grading.

Example: For “What is polymorphism in OOP?”, an answer claiming only “one class can have many objects” is partial. Feedback should explain that polymorphism allows a common interface or method to have different implementations, and review method overriding/runtime polymorphism.

6. Machine Learning and Adaptive Analytics

Technique	Use
Rule-based engine	Initial adaptive selection and transparent decisions
Bayesian Knowledge Tracing	Estimate mastery of a concept across attempts
Logistic Regression	Predict probability of correct response
Random Forest	Predict weak-topic or performance category
K-Means	Explore learner clusters; use cautiously
Exponential moving average	Update mastery from recent practice
Pandas / NumPy	Feature preparation and descriptive statistics

- Features: per-topic accuracy, response time, attempt count, difficulty, repeated mistakes, score trend, hint usage, and confidence compared with correctness.
- Start with rule-based selection and simple mastery estimates; avoid training models from scratch at launch.
- Keep recommendations explainable and based on learning activity; avoid sensitive profiling.
- Validate with held-out data and human-graded examples before production use.

7. Gamification

Action	Illustrative XP
Correct Easy answer	10
Correct Medium answer	20
Correct Hard answer	30
Correct Expert answer	50
Perfect answer bonus	+10
Complete 10-question session	+50
Daily quest completion	+100
Weak-topic mastery improvement	+75
Revision challenge completion	+40

Do not deduct XP for incorrect answers. Award partial learning XP for reviewing feedback and answering a corrective follow-up. XP values are configurable balancing defaults.

Rank	Level range	Focus
Beginner	1–5	Fundamentals
Intermediate	6–10	Applied concepts
Advanced	11–20	Complex and scenario questions
Master	21+	Consistent topic mastery

Badge	Condition
First Step	Complete first practice session
Quick Learner	Answer 10 questions correctly
Perfectionist	100% session accuracy
Streak Master	Practice 7 consecutive days
Comeback Hero	Improve prior score by 20%
Topic Master	Reach 90% topic mastery
Boss Slayer	Complete Expert challenge
Knowledge Explorer	Practice 10 topics

- Daily quest examples: answer five correctly; revise three mistakes; finish a mock interview; improve the weakest topic; complete a 15-minute practice.
- Skill-tree example: Programming Fundamentals → Python/C Basics → OOP → Data Structures → Algorithms → Technical Interview.
- Calculate rewards on the server, use idempotency keys, and prevent duplicate quest claims. Leaderboards should be optional and privacy-aware.

8. Redis Caching and Optimization

Cache	Example key	Suggested TTL
Topic analysis	topic_analysis:{content_hash}	24 hours
Question generation	questions:{content_hash}:{config_hash}	12 hours
Reference answer	reference:{question_hash}	24 hours
Evaluation	evaluation:{question_hash}:{answer_hash}:{rubric_version}	1 hour
Session analytics	analytics:{session_id}	30 minutes
Daily quest	quest:{anonymous_id}:{date}	24 hours
Daily leaderboard	leaderboard:daily	5 minutes
Rate limit	rate_limit:{anonymous_id}	1 minute

Cache flow: request → Redis lookup → cache hit returns result; cache miss invokes AI → validate output → cache safe reusable result → persist important records in MongoDB → return response.

- Normalize content before hashing; include model, prompt, and rubric versions in keys.
- Use Redis locks to reduce duplicate concurrent AI calls.
- Cache exact answer evaluations carefully; broad semantic cache reuse may transfer incorrect judgments.
- Do not share personalized evaluations across users unless the result is demonstrably user-independent.
- Define TTL and invalidation for every temporary key.

9. Anonymous Access, Persistence, and Privacy

Generate a random browser anonymous identifier (for example, crypto.randomUUID()) and store it in IndexedDB or localStorage. Store practice progress in IndexedDB. MongoDB may hold optional anonymous cloud records.

- Allow users to begin immediately without account creation.
- Provide progress export/import and reset controls.
- Browser data can be lost if storage is cleared or a user changes device.
- Optional cross-device continuity can use a recovery token/backup code; explain it is not a standard account.
- An anonymous ID is not authentication. Validate session ownership and rate-limit APIs.
- Minimize retained content and attempts; document retention and deletion policies.

10. High-Level Design (HLD)

Architecture: Browser (React/Vite) → FastAPI API → modular services (content analyzer, question generator, evaluator, analytics, adaptive engine, gamification) → Redis (cache/rate limits/locks/leaderboard) and MongoDB (durable records). Backend AI modules call the AI provider. An optional background worker processes heavy jobs.

```
User / Browser
■■ React UI, client state, IndexedDB
■■ HTTPS REST API
■■ FastAPI Backend
■■ Content Analyzer ■■ AI Provider
■■ Question Generator ■■ AI Provider
■■ Answer Evaluator ■■ AI Provider
■■ Analytics / ML / Adaptive Engine
■■ Gamification Engine
■■ Redis: cache, limits, locks, leaderboard
■■ MongoDB: durable application records
■■ Optional Background Worker
```

Use a modular monolith first. Keep AI credentials server-side and avoid microservices until scale or deployment requirements justify them.

11. Low-Level Design (LLD)

Service file	Responsibility
content_service.py	Extract, clean, and analyze content
topic_service.py	Detect concepts and topic relationships
question_service.py	Generate and manage questions
evaluation_service.py	Evaluate responses and create corrections
analytics_service.py	Compute learner statistics
ml_service.py	Estimate mastery and difficulty
adaptive_service.py	Choose the next question
gamification_service.py	Manage XP, badges, streaks, quests
cache_service.py	Redis caching and locks
session_service.py	Anonymous sessions
repositories / storage_service.py	MongoDB persistence

```
EvaluationRequest(BaseModel):
question_id: str
user_answer: str
session_id: str

EvaluationResult(BaseModel):
score: float
rating: Literal[Excellent, Good, Needs Improvement, Weak]
correct_answer: str
missing_concepts: list[str]
improvement_suggestions: list[str]
confidence: float

Service sequence:
1. Retrieve question and rubric
2. Check Redis cache
3. Evaluate on cache miss
4. Validate structured output
5. Save evaluation in MongoDB
6. Update analytics and mastery
7. Award XP idempotently
8. Return feedback
```

12. MongoDB Database Schema

Collection	Key fields / purpose
content	_id, session_id, title, content_hash, topics[], created_at
questions	_id, content_id, topic, question, difficulty, type, reference_answer, rubric
practice_sessions	_id, anonymous_id, content_id, total_questions, completed_questions, accuracy, total_xp, status, created_at
answer_attempts	_id, session_id, question_id, user_answer, score, missing_concepts[], confidence, response_time_ms, created_at
gamification	anonymous_id, xp, level, streak, badges[], completed_quests[], updated_at
topic_mastery	anonymous_id, topic_id, mastery_estimate, attempts, updated_at
daily_quests	quest_id, quest_definition, date, reward_xp
leaderboard_entries	anonymous alias, points, period
content_chunks	content_id, chunk_id, text, topic, hash
model_evaluations	model_version, prompt_version, test_case, expected_score, observed_score

Create indexes for session_id, anonymous_id, content_hash, question_id, and timestamps. Use TTL indexes only for temporary data. Establish retention and deletion rules.

13. API Design (REST / Postman Friendly)

Method	Endpoint	Purpose
POST	/api/v1/content/analyze	Analyze submitted text
POST	/api/v1/content/upload	Upload PDF/DOCX
GET	/api/v1/content/{content_id}/topics	Get detected topics
POST	/api/v1/questions/generate	Generate questions
GET	/api/v1/questions/{question_id}	Get question details
POST	/api/v1/sessions/start	Start anonymous session
POST	/api/v1/sessions/{session_id}/submit	Submit answer for evaluation
GET	/api/v1/sessions/{session_id}/result	Get session result
GET	/api/v1/analytics/{session_id}	Get analytics
GET	/api/v1/analytics/{session_id}/weak-topics	Get weak topics
GET	/api/v1/analytics/{session_id}/recommendations	Get learning suggestions
GET	/api/v1/gamification/{anonymous_id}	Get XP, levels, badges
GET	/api/v1/gamification/quests	Get daily quests
POST	/api/v1/gamification/quests/{quest_id}/claim	Claim quest reward
GET	/api/v1/leaderboard/daily	Get daily leaderboard
POST	/api/v1/progress/export	Export progress
POST	/api/v1/progress/import	Import progress
DELETE	/api/v1/progress/{anonymous_id}	Delete progress

```
Question generation request:
{
"content_id": "content_123",
"topics": ["OOP", "Inheritance", "Polymorphism"],
"question_count": 10,
"difficulty": "medium",
"question_types": ["descriptive", "mcq", "technical"],
"adaptive_mode": true
}

Answer submission:
{
"session_id": "session_123",
"question_id": "question_456",
"user_answer": "Polymorphism means ...",
"response_time_ms": 45000
}

Evaluation response:
{
"success": true,
"data": {
"score": 82, "rating": "Good",
"correct_answer": "Polymorphism allows ...",
"missing_concepts": ["Runtime polymorphism"],
"improvement_suggestions": ["Practice overriding"],
"xp_earned": 25, "next_difficulty": "medium"
}
}
```

14. Frontend UI/UX Plan

Page	Main UI elements
Dashboard	XP, level, streak, mastery, recent sessions, quick start
Content Input	Topic/text/file upload; count, type, difficulty configuration
Practice Arena	Question, topic, difficulty, timer, progress, answer editor, hints, submit
AI Feedback	Score, rating, corrected answer, missing concepts, suggestions, XP animation
Analytics	Accuracy, mastery, score trend, weak topics, response-time charts
Gamification Hub	Badges, quests, leaderboard, XP, levels, skill tree
Settings	Privacy, export/import, reset, retention controls

- Design for desktop, tablet, and mobile.
- Suggested palette: dark navy with violet/cyan accents; keep glass effects restrained.
- Use clear visual states for correct, partial, and incorrect answers.
- Include loading, error, retry, empty, and offline states.
- Practice arena includes question progress, answer area, optional hint, timer, feedback, next question, and reward animation.

15. Exact Directory Structure

```
skillarena-ai/
■■■ frontend/
■ ■■■ public/{icons,badges}/
■ ■■■ src/
■ ■ ■■■ app/{App.tsx,routes.tsx,providers.tsx}
■ ■ ■■■ components/{ui,dashboard,practice,feedback,analytics,gamification}/
■ ■ ■■■ pages/{Dashboard,ContentInput,PracticeArena,Feedback,Analytics,Achievements,Settings}.tsx
■ ■ ■■■ hooks/
■ ■ ■■■ services/{api,content,questions,analytics,gamification}.ts
■ ■ ■■■ store/{sessionStore,gamificationStore}.ts
■ ■ ■■■ utils/
■ ■ ■■■ types/
■ ■ ■■■ styles/
■ ■■■ package.json
■ ■■■ vite.config.ts
■■■ backend/
■ ■■■ app/{main.py,config.py}
■ ■■■ app/api/v1/{content,questions,sessions,analytics,gamification,leaderboard}.py
■ ■■■ app/core/{security,exceptions,logging}.py
■ ■■■ app/services/{content_service,topic_service,question_service,evaluation_service,analytics_service,adaptive_service,
gamification_service}.py
■ ■■■ app/ai/{provider,prompt_builder,evaluator,generator,output_validator}.py
■ ■■■ app/ml/{feature_engineering,mastery_model,difficulty_model,recommendation_model}.py
■ ■■■ app/cache/{redis_client,cache_service}.py
■ ■■■ app/database/{mongodb.py,repositories/}
■ ■■■ app/schemas/
■ ■■■ app/utils/
■ ■■■ app/workers/
■ ■■■ tests/{unit,integration,evaluation}/
■ ■■■ requirements.txt
■ ■■■ Dockerfile
■■■ infrastructure/{docker-compose.yml,redis.conf}
■■■ docs/{HLD.md,LLD.md,API.md,DATABASE.md}
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

16. AI Prompt Blueprint

```
SYSTEM ROLE: AI educational assessment agent.
OBJECTIVE: Evaluate the user response using supplied source content, question, reference answer, and rubric.
RULES:
1. Assess meaning and conceptual correctness, not exact wording.
2. List correct, incorrect, and missing concepts.
3. Do not invent unsupported facts.
4. Disclose ambiguity or insufficient source evidence.
5. Give a corrected answer and actionable suggestions.
6. Apply the rubric consistently; do not reward filler.
7. Return structured JSON with score, rating, correct_concepts, incorrect_concepts, missing_concepts, corrected_answer,
improvement_suggestions, confidence.
```

Validate model output using Pydantic. If malformed, retry once with a repair prompt or return a safe error. Store prompt, model, and rubric versions for auditing.

17. Security and Performance

Requirement	Implementation
API security	HTTPS, validation, CORS allowlist

Requirement	Implementation
Abuse prevention	Redis rate limits and quotas
AI cost controls	Per-session quotas, token limits, model fallback
File safety	MIME validation, size limit, safe parsing
Prompt injection	Treat uploaded content as untrusted; delimit source text
Privacy	Minimize storage; retention/deletion controls
Session protection	Check session ownership; anonymous ID is not auth
Database	MongoDB authentication, private network, least privilege
Redis	Authentication/private network; never expose publicly
Secrets	Environment variables or secret manager
Output	Validate JSON; sanitize rendered Markdown/HTML
Rewards	Server-side XP and idempotent claims

- Suggested MVP file limit: 10 MB; maximum generated questions per request: 20.
- Set answer length limits, AI timeouts, and per-session daily quotas.
- Provide safe fallback messages for provider quota, timeout, and service failures.

18. Deployment Architecture

Component	Suggested option
Frontend	Vercel, Netlify, or Cloudflare Pages
Backend	Render, Railway, or cloud VM
Database	MongoDB Atlas
Cache	Redis Cloud
AI provider	Gemini, OpenAI, or compatible provider
Worker	Same backend initially; separate process later
Monitoring	Sentry and structured logging

```
User → CDN / Frontend Hosting → React Vite
React → HTTPS → FastAPI Backend
FastAPI → AI Provider
FastAPI → Redis Cloud
FastAPI → MongoDB Atlas
FastAPI → Optional Background Worker
```

Keep provider credentials on the backend. Docker supports repeatable development and deployment. Begin with a single backend deployment; add a worker for heavy tasks when needed.

19. Eight-Week Implementation Roadmap

Phase	Week	Deliverables
Setup	1	React/Vite, FastAPI, MongoDB, Redis, env config, API skeleton
Content analysis	2	Text/topic input, PDF/DOCX extraction, cleaning, topics, hashing
Question generation	3	AI integration, types/difficulty, answer keys/rubrics, duplicate detection
Evaluation	4	Semantic grading, scores, corrections, feedback, suggestions
Gamification	5	XP, levels, badges, quests, streaks, skill tree, leaderboard
Analytics and ML	6	Pandas/NumPy, weak-topic analytics, adaptive difficulty, mastery
UI and optimization	7	Responsive UI, animations, Redis optimization, local persistence
Testing/deployment	8	Unit, integration, evaluation, security, performance tests and deployment

This is an estimate for one developer working consistently. A polished production release may require additional time.

20. Success Metrics and Test Plan

Metric	Initial target (aspirational)
Non-AI API latency	< 300 ms p95
Redis cache response	< 100 ms p95
Question generation	< 10 seconds target
Answer evaluation	< 8 seconds target
Availability	99.5% MVP target

Metric	Initial target (aspirational)
Duplicate question rate	< 3%
Human grader agreement	≥ 85% on curated set
Grounded evaluation correctness	≥ 95% on curated tests
Gamification reward logic	100% automated test pass
Learning progress	Measure score change across repeated sessions

- Build a curated question-answer test set with human grades.
- Test consistency, hallucinations, ambiguous content, edge cases, and prompt injection.
- Test Redis/AI outage, timeout, quota limits, malformed output, and database failure.
- Targets are planning goals, not guaranteed outcomes; revise after measurement.

21. MVP Scope and Priorities

Recommended build order: content input and topic analysis → question generation → accurate evaluation → feedback and corrections → anonymous persistence → gamification → analytics/adaptive ML → production caching and optimization.

- MVP essentials: content entry, topic selection, question generation, answer submission, feedback/rating, session score, local progress, basic XP/levels, and Redis cache.
- Do not begin with complex ML or leaderboard features. Validate question quality and evaluation reliability first.
- Later releases: skill trees, advanced ML, leaderboards, spaced repetition, voice interviews, and company-facing features.

22. Product Journey and Final Vision

Core learning loop: user content → AI questions → practice → answer evaluation → corrections and feedback → performance analysis → XP/rewards → adaptive learning → next practice.

SkillArena AI turns learning material into a structured, personalized interview preparation experience. It finds gaps, explains mistakes, and selects subsequent practice from observed performance. AI grading is not perfect; source grounding, transparent rubrics, human validation, privacy, and fair reward logic remain essential.